

SQL Mini Project

Retail Database Analysis

Submitted by:

Tanmay Mandal

Database: MySQL | Project Type: Data Analysis With GenAI

Year: 2025– 2026

1. Introduction

This document presents a structured SQL Mini Project built on a simulated Retail database. The project demonstrates practical application of SQL — from basic data retrieval to advanced multi-table joins, subqueries, and set operations.

The Retail database models a real-world e-commerce environment, capturing the full lifecycle of a transaction: from customer registration, product cataloguing, and order placement through to payment processing and post-purchase product reviews.

All queries have been written for MySQL and are organised into six progressive difficulty levels, making this project suitable as both a learning exercise and a portfolio demonstration.

2. Purpose

- The primary objectives of this project are:
- 1. To practise and consolidate SQL skills across all major query categories.
 - 2. To analyse retail business data and derive meaningful insights.
 - 3. To demonstrate proficiency in data filtering, aggregation, multi-table joins, subqueries, and set operations.
 - 4. To produce a well-documented reference document that can be used for academic submission or portfolio review.

3. Database Overview

Database Name: Retail
Platform: MySQL
Total Tables: 6

Tables at a Glance

Column	Data Type	Description
customers	6 columns	Stores customer identity and contact data
products	6 columns	Product catalogue with category, price, and stock
orders	5 columns	Records each order placed by a customer
order_items	5 columns	Line-item detail linking orders to products
payments	5 columns	Payment records per order with method details
product_reviews	6 columns	Customer ratings and review text per product

4. Entity Relationship Diagram (ERD)

The diagram below illustrates the relationships between all six tables in the Retail database. Primary keys (PK) and foreign key (FK) links are shown.

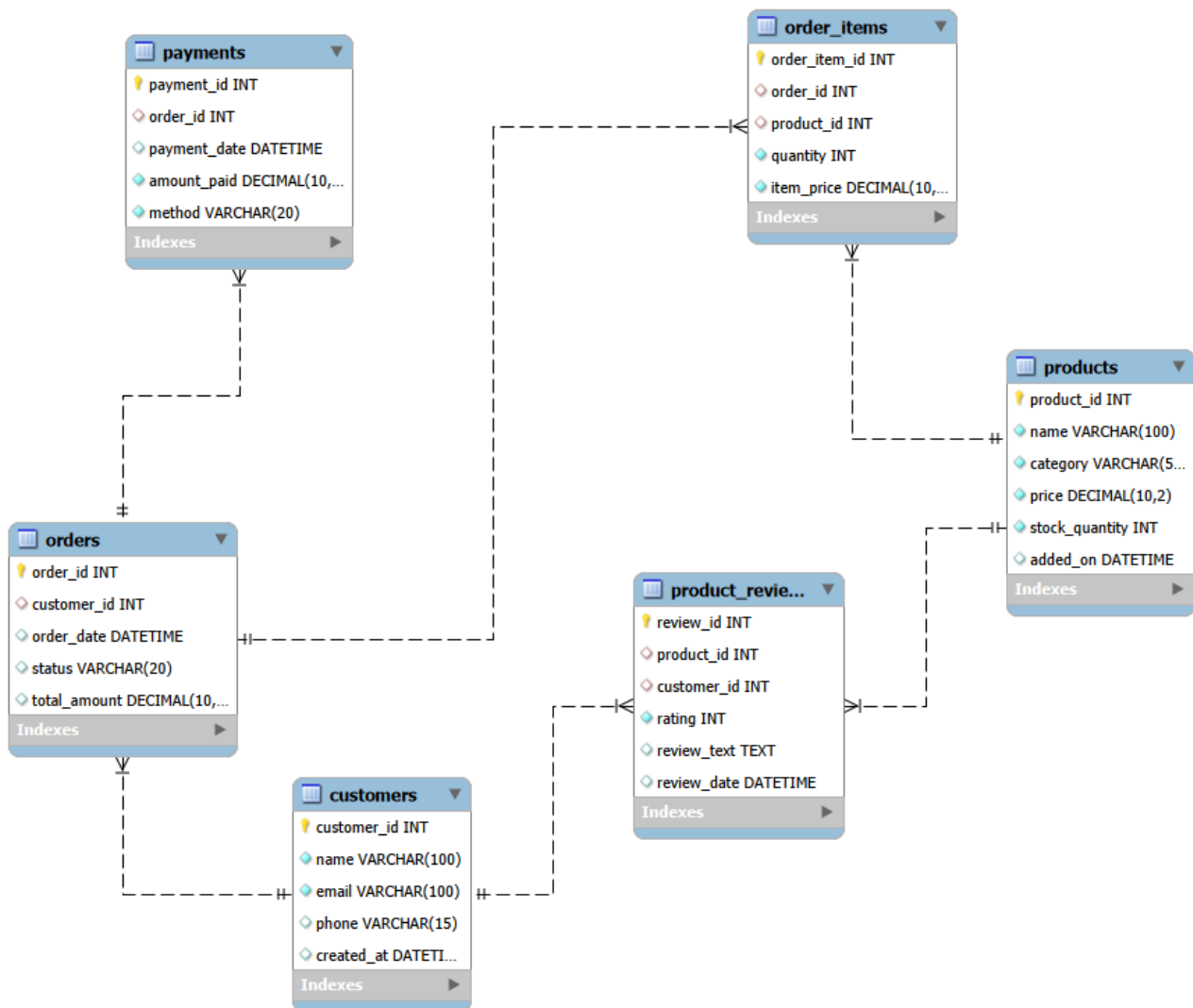


Figure 1 – Entity Relationship Diagram: Retail Database

5. Table Column Reference

customers

Column	Data Type	Description
customer_id	INT (PK, AI)	Unique identifier for each customer
name	VARCHAR(100)	Full name of the customer
email	VARCHAR(100)	Email address (UNIQUE)
phone	VARCHAR(15)	Contact phone number
created_at	DATETIME	Account creation timestamp (default: NOW)

products

Column	Data Type	Description
product_id	INT (PK, AI)	Unique identifier for each product
name	VARCHAR(100)	Product name
category	VARCHAR(50)	Product category (Electronics, Books, etc.)
price	DECIMAL(10,2)	Selling price in INR
stock_quantity	INT	Units currently in stock
added_on	DATETIME	Date/time product was added to catalogue

orders

Column	Data Type	Description
order_id	INT (PK, AI)	Unique order identifier
customer_id	INT (FK)	References customers.customer_id
order_date	DATETIME	Timestamp when order was placed
status	VARCHAR(20)	Order status: Pending / Shipped / Delivered / Cancelled
total_amount	DECIMAL(10,2)	Gross order value in INR

order_items

Column	Data Type	Description
order_item_id	INT (PK, AI)	Unique line-item identifier
order_id	INT (FK)	References orders.order_id
product_id	INT (FK)	References products.product_id
quantity	INT	Number of units ordered (> 0)
item_price	DECIMAL(10,2)	Unit price at time of order

payments

Column	Data Type	Description
payment_id	INT (PK, AI)	Unique payment record identifier
order_id	INT (FK)	References orders.order_id
payment_date	DATETIME	Timestamp of payment
amount_paid	DECIMAL(10,2)	Amount received (> 0)
method	VARCHAR(20)	Payment mode: UPI / Credit Card / Debit Card / Net Banking

product_reviews

Column	Data Type	Description
review_id	INT (PK, AI)	Unique review identifier
product_id	INT (FK)	References products.product_id
customer_id	INT (FK)	References customers.customer_id
rating	INT	Star rating given by customer
review_text	TEXT	Written review content
review_date	DATETIME	Timestamp of review submission

6. SQL Queries with Answers

Level 1: Basics

Q1. Retrieve customer names and emails for email marketing

Columns Used: *customers: name, email*

```
SELECT name, email FROM customers;
```

Output:

	name	email
▶	Thomas Owens	user1@example.com
	Charles Grant	user2@example.com
	Kaitlin Richards	user3@example.com
	Christina Williams	user4@example.com
	David Allen	user5@example.com
	Mark Duke	user6@example.com
	Briana Wright	user7@example.com
	John Bryan	user8@example.com
	Jason Thompson	user9@example.com
	Shawn Hill	user10@example.com
	Walter Jenkins	user11@example.com
	Mary Knight	user12@example.com
	Leslie Wilson	user13@example.com

Q2. View complete product catalogue with all available details

Columns Used: *products: all columns*

```
SELECT * FROM products;
```

Output:

	product_id	name	category	price	stock_quantity	added_on
▶	1	Plant No	Home	639.43	152	2024-01-30 06:30:53
	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38

Q3. List all unique product categories**Columns Used:** *products: category*

```
SELECT DISTINCT category FROM products;
```

Output:

category
Home
Clothing
Electronics
Toys
Books

Q4. Show all products priced above ₹1,000**Columns Used:** *products: all columns (filtered by price)*

```
SELECT * FROM products WHERE price > 1000;
```

Output:

	product_id	name	category	price	stock_quantity	added_on
▶	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53

Q5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)**Columns Used:** *products: all columns*

```
SELECT * FROM products WHERE price BETWEEN 2000 AND 5000;
```

Output:

	product_id	name	category	price	stock_quantity	added_on
▶	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
	13	Full West	Books	2112.33	172	2023-09-15 03:13:38
	17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
	18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
	20	Real Source	Books	4398.66	197	2025-02-08 10:28:27

Q6. Fetch data for specific customer IDs (e.g., from loyalty program list)**Columns Used:** *customers: all columns (filtered by customer_id)*

```
SELECT * FROM customers WHERE customer_id IN (1, 5, 10);
```

Output:

	customer_id	name	email	phone	created_at
▶	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 1 2024-10-
	5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
	10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
•	NULL	NULL	NULL	NULL	NULL

Q7. Identify customers whose names start with the letter 'A'**Columns Used:** *customers: name*

```
SELECT name FROM customers WHERE name LIKE 'A%';
```

Output:

	customer_id	name	email	phone	created_at
▶	15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
	16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
	19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
	27	Adrienne Green	user27@example.com	530.644.8455x93	2023-08-22 01:55:29
•	NULL	NULL	NULL	NULL	NULL

Q8. List electronics products priced under ₹3,000**Columns Used:** *products: all columns (filtered by category, price)*

```
SELECT * FROM products
WHERE category = 'Electronics' AND price < 3000;
```

Output:

	product_id	name	category	price	stock_quantity	added_on
▶	3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
	6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
	15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
	31	Series Page	Electronics	2070.37	83	2024-04-01 00:24:06
	34	Despite Win	Electronics	1340.34	64	2024-11-27 06:55:45
	44	Actually Term	Electronics	396.11	85	2023-11-02 13:09:20
	47	Southern Thing	Electronics	512.46	40	2024-02-28 17:57:38
•	NULL	NULL	NULL	NULL	NULL	NULL

Q9. Display product names and prices in descending order of price**Columns Used:** *products: name, price*

```
SELECT name, price FROM products ORDER BY price DESC;
```

Output:

	name	price
▶	Response Indeed	4897.36
	Population Social	4813.68
	Development System	4801.78
	Any Question	4759.28
	Fire Often	4734.89
	Natural Network	4722.66
	Build High	4707.14
	Serious Recognize	4523.10
	Study Total	4413.68
	Real Source	4398.66
	Involve Officer	4244.09
	Group But	4180.23
	Every Amount	4173.60

Q10. Display product names and prices, sorted by price then by name**Columns Used:** *products: name, price*

```
SELECT name, price FROM products ORDER BY price DESC, name ASC;
```

Output:

	name	price
▶	Response Indeed	4897.36
	Population Social	4813.68
	Development System	4801.78
	Any Question	4759.28
	Fire Often	4734.89
	Natural Network	4722.66
	Build High	4707.14
	Serious Recognize	4523.10
	Study Total	4413.68
	Real Source	4398.66
	Involve Officer	4244.09
	Group But	4180.23
	Every Amount	4173.60

Level 2: Filtering and Formatting

Q1. Retrieve orders where customer information is missing

Columns Used: *orders: all columns*

```
SELECT * FROM orders WHERE customer_id IS NULL;
```

Output:

	order_id	customer_id	order_date	status	total_amount
*	NULL	NULL	NULL	NULL	NULL

Q2. Display customer names and emails using column aliases for frontend readability

Columns Used: *customers: name, email*

```
SELECT name AS "Customer Name",  
       email AS "Email Address" FROM customers;
```

Output:

	Customer Name	Email Address
▶	Thomas Owens	user1@example.com
	Charles Grant	user2@example.com
	Kaitlin Richards	user3@example.com
	Christina Williams	user4@example.com
	David Allen	user5@example.com
	Mark Duke	user6@example.com
	Briana Wright	user7@example.com
	John Bryan	user8@example.com
	Jason Thompson	user9@example.com
	Shawn Hill	user10@example.com
	Walter Jenkins	user11@example.com
	Mary Knight	user12@example.com

Q3. Calculate total value per item ordered by multiplying quantity and item price

Columns Used: *order_items: order_item_id, order_id, product_id, quantity, item_price*

```
SELECT order_item_id, order_id, product_id, quantity,  
       item_price, quantity * item_price AS total_item_value  
FROM order_items;
```

Output:

	order_item_id	order_id	product_id	quantity	item_price	total_item_value
▶	1	1	19	2	4707.14	9414.28
	2	2	6	3	177.40	532.20
	3	3	9	3	985.19	2955.57
	4	3	23	1	2208.99	2208.99
	5	4	35	2	4734.89	9469.78
	6	5	19	1	4707.14	4707.14
	7	5	7	2	4897.36	9794.72
	8	6	7	3	4897.36	14692.08
	9	6	37	1	1429.45	1429.45
	10	6	15	1	723.97	723.97
	11	6	35	3	4734.89	14204.67
	12	7	30	1	3043.67	3043.67
	13	8	10	3	4801.78	14405.34

Q4. Combine customer name and phone number in a single column**Columns Used:** *customers: name, phone*

```
SELECT CONCAT(name, ' - ', phone) AS customer_contact
FROM customers;
```

Output:

	customer_contact
▶	Thomas Owens - 142-479-1945
	Charles Grant - 9153947511
	Kaitlin Richards - 2073473421
	Christina Williams - 586-605-5061x06
	David Allen - (751)456-8289x1
	Mark Duke - (144)957-2811
	Briana Wright - 223-833-9635
	John Bryan - 045.568.0798x27
	Jason Thompson - 1862659420
	Shawn Hill - (268)113-3152x7
	Walter Jenkins - 536-329-0817x71
	Mary Knight - 361-636-3802
	Leslie Wilson - +1-256-261-1984

Q5. Extract only the date part from order timestamps for date-wise reporting**Columns Used:** *orders: order_id, customer_id, order_date, status, total_amount*

```
SELECT order_id, customer_id, DATE(order_date) AS order_date_only,
       status, total_amount FROM orders;
```

Output:

	order_id	customer_id	order_date_only	status	total_amount
▶	1	20	2025-03-02	Delivered	9414.28
	2	18	2024-10-09	Shipped	532.20
	3	15	2025-05-08	Cancelled	5164.56
	4	11	2024-09-19	Delivered	9469.78
	5	12	2025-04-08	Pending	14501.86
	6	29	2024-10-25	Cancelled	31050.17
	7	22	2024-07-29	Shipped	3043.67
	8	19	2024-07-30	Cancelled	32714.06
	9	6	2025-06-10	Pending	24219.20
	10	28	2025-02-16	Delivered	24342.52
	11	25	2025-03-11	Cancelled	16196.13
	12	28	2025-01-15	Shipped	9890.72
	13	25	2025-05-12	Cancelled	9627.36

Q6. List products that do not have any stock left**Columns Used:** *products: all columns (filtered by stock_quantity)*

```
SELECT * FROM products WHERE stock_quantity = 0;
```

Output:

	product_id	name	category	price	stock_quantity	added_on
•	NULL	NULL	NULL	NULL	NULL	NULL

Level 3: Aggregations

Q1. Count the total number of orders placed

Columns Used: *orders: order_id*

```
SELECT COUNT(*) AS total_orders FROM orders;
```

Output:

	total_orders
▶	400

Q2. Calculate the total revenue collected from all orders

Columns Used: *orders: total_amount*

```
SELECT SUM(total_amount) AS total_revenue FROM orders;
```

Output:

	total_revenue
▶	6960973.66

Q3. Calculate the average order value

Columns Used: *orders: total_amount*

```
SELECT ROUND(AVG(total_amount), 2) AS avg_order_value FROM orders;
```

Output:

	avg_order_value
▶	17402.43

Q4. Count customers who have placed at least one order

Columns Used: *order_items: order_id, quantity* | *orders: order_id, customer_id* | *customers: customer_id*

```
SELECT COUNT(DISTINCT customer_id) AS count_with_orders  
FROM orders  
WHERE customer_id IS NOT NULL;
```

Output:

	count_with_orders
▶	30

Q5. Find the number of orders placed by each customer**Columns Used:** *orders: customer_id, order_id*

```
SELECT customer_id, COUNT(order_id) AS total_orders
FROM orders
GROUP BY customer_id
ORDER BY total_orders DESC;
```

Output:

	customer_id	total_orders
▶	29	20
	2	17
	3	17
	17	17
	20	17
	28	17
	6	16
	16	16
	30	16
	14	15
	15	15
	19	15
	22	15

Q6. Find total sales amount made by each customer**Columns Used:** *orders: customer_id, total_amount*

```
SELECT customer_id, SUM(total_amount) AS total_sales
FROM orders
GROUP BY customer_id
ORDER BY total_sales DESC;
```

Output:

	customer_id	total_sales
▶	29	435408.89
	24	379286.82
	28	362584.19
	14	360324.18
	25	335100.94
	2	284420.07
	30	281841.38
	16	278469.73
	5	262504.19
	17	262189.71
	15	260499.91
	3	253783.31
	10	252722.75

Q7. List the number of products sold per category**Columns Used:** *order_items: order_item_id, product_id | products: category*

```
SELECT p.category,
       SUM(oi.quantity) AS units_sold
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.category
ORDER BY units_sold DESC;
```

Output:

	category	units_sold
▶	Electronics	687
	Clothing	559
	Home	443
	Toys	405
	Books	350

Q8. Find the average item price per category**Columns Used:** *products: category, price*

```
SELECT p.category, ROUND(AVG(p.price), 2) AS avg_price
FROM products p
GROUP BY p.category ORDER BY avg_price DESC;
```

Output:

	category	avg_price
▶	Clothing	3434.58
	Books	3167.08
	Electronics	2653.39
	Toys	2516.53
	Home	2146.37

Q9. Show number of orders placed per day**Columns Used:** *orders: order_date, order_id*

```
SELECT DATE(order_date) AS order_day, COUNT(order_id) AS orders_count
FROM orders GROUP BY DATE(order_date) ORDER BY order_day;
```

Output:

	order_day	orders_count
▶	2024-06-15	1
	2024-06-18	1
	2024-06-19	3
	2024-06-20	2
	2024-06-21	2
	2024-06-23	2
	2024-06-24	1
	2024-06-25	1
	2024-06-30	2
	2024-07-03	1
	2024-07-04	1
	2024-07-05	3
	2024-07-06	2

Q10. List total payments received per payment method**Columns Used:** *payments: method, amount_paid*

```
SELECT method, ROUND(SUM(amount_paid), 2) AS total_received  
FROM payments  
GROUP BY method ORDER BY total_received DESC;
```

Output:

	method	total_received
►	Debit Card	1930577.88
	Credit Card	1754603.10
	Net Banking	1658383.90
	UPI	1617408.78

Level 4: Multi-Table Queries (JOINS)

Q1. Retrieve order details along with the customer name (INNER JOIN)

Columns Used: *orders: order_id, customer_id, order_date, status, total_amount* | *customers: name*

```
SELECT o.order_id, c.name AS customer_name, o.order_date,
       o.status, o.total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

Output:

	order_id	customer_name	order_date	status	total_amount
▶	14	Thomas Owens	2024-09-24 21:21:38	Shipped	15803.34
	17	Thomas Owens	2024-08-19 21:17:57	Pending	11173.77
	61	Thomas Owens	2024-12-26 23:21:58	Shipped	13053.00
	76	Thomas Owens	2025-01-14 22:59:54	Pending	23506.81
	92	Thomas Owens	2024-09-26 11:33:58	Shipped	834.75
	109	Thomas Owens	2025-03-17 04:56:18	Shipped	12190.14
	127	Thomas Owens	2025-06-10 18:01:32	Pending	20160.64
	135	Thomas Owens	2025-03-11 22:58:23	Pending	8891.79
	144	Thomas Owens	2024-09-19 09:18:22	Shipped	12093.54
	221	Thomas Owens	2024-09-25 05:49:39	Pending	12437.61
	278	Thomas Owens	2025-02-01 20:05:38	Delivered	32015.16
	379	Thomas Owens	2025-03-22 15:57:58	Cancelled	21586.89
	56	Charles Grant	2024-06-23 15:59:57	Pending	6828.61

Q2. Get list of products that have been sold (INNER JOIN with order_items)

Columns Used: *products: product_id, name, category, price* | *order_items: product_id*

```
SELECT DISTINCT p.product_id, p.name AS product_name, p.category, p.price
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id;
```

Output:

	product_id	product_name	category	price
▶	1	Plant No	Home	639.43
	2	Population Social	Clothing	4813.68
	3	Available Answer	Electronics	2529.51
	4	Any Question	Clothing	4759.28
	5	Natural Network	Toys	4722.66
	6	If Whatever	Electronics	177.40
	7	Response Indeed	Clothing	4897.36
	8	Every Amount	Home	4173.60
	9	Common Study	Toys	985.19
	10	Development System	Electronics	4801.78
	11	Build Her	Books	1852.64
	12	Action Ask	Electronics	4017.01
	13	Full West	Books	2112.33

Q3. List all orders with their payment method (INNER JOIN)**Columns Used:** *orders: order_id, order_date, total_amount | payments: method, amount_paid*

```
SELECT o.order_id, o.order_date, o.total_amount,
       pay.method, pay.amount_paid
FROM orders o
INNER JOIN payments pay ON o.order_id = pay.order_id;
```

Output:

	order_id	order_date	total_amount	method	amount_paid
▶	1	2025-03-02 07:20:11	9414.28	Credit Card	9414.28
	2	2024-10-09 18:08:21	532.20	Net Banking	532.20
	3	2025-05-08 00:08:27	5164.56	Credit Card	5164.56
	4	2024-09-19 22:16:13	9469.78	UPI	9469.78
	5	2025-04-08 18:02:06	14501.86	UPI	14501.86
	6	2024-10-25 07:33:59	31050.17	UPI	31050.17
	7	2024-07-29 11:58:47	3043.67	UPI	3043.67
	8	2024-07-30 22:49:49	32714.06	Net Banking	32714.06
	9	2025-06-10 17:00:25	24219.20	Net Banking	24219.20
	10	2025-02-16 12:45:59	24342.52	Debit Card	24342.52
	11	2025-03-11 14:22:46	16196.13	Credit Card	16196.13
	12	2025-01-15 13:22:53	9890.72	Credit Card	9890.72
	13	2025-05-12 13:04:29	9627.36	UPI	9627.36

Q4. Get list of customers and their orders (LEFT JOIN)**Columns Used:** *customers: customer_id, name | orders: order_id, order_date, total_amount*

```
SELECT c.customer_id, c.name, o.order_id, o.order_date, o.total_amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
ORDER BY c.customer_id;
```

Output:

	customer_id	name	order_id	order_date	total_amount
▶	1	Thomas Owens	14	2024-09-24 21:21:38	15803.34
	1	Thomas Owens	17	2024-08-19 21:17:57	11173.77
	1	Thomas Owens	61	2024-12-26 23:21:58	13053.00
	1	Thomas Owens	76	2025-01-14 22:59:54	23506.81
	1	Thomas Owens	92	2024-09-26 11:33:58	834.75
	1	Thomas Owens	109	2025-03-17 04:56:18	12190.14
	1	Thomas Owens	127	2025-06-10 18:01:32	20160.64
	1	Thomas Owens	135	2025-03-11 22:58:23	8891.79
	1	Thomas Owens	144	2024-09-19 09:18:22	12093.54
	1	Thomas Owens	221	2024-09-25 05:49:39	12437.61
	1	Thomas Owens	278	2025-02-01 20:05:38	32015.16
	1	Thomas Owens	379	2025-03-22 15:57:58	21586.89
	2	Charles Grant	56	2024-06-23 15:59:57	6828.61

Q5. List all products along with order item quantity (LEFT JOIN)**Columns Used:** *products: product_id, name, category | order_items: product_id, quantity*

```

SELECT p.product_id, p.name, p.category,
       IFNULL(SUM(oi.quantity), 0) AS total_quantity_sold
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name, p.category
ORDER BY total_quantity_sold DESC;

```

Output:

	product_id	product_name	category	total_quantity_sold
▶	24	Assume Serve	Home	83
	46	A Age Treatment	Home	67
	18	Some Them	Toys	63
	47	Southern Thing	Electronics	62
	8	Every Amount	Home	61
	22	Move Small	Books	60
	20	Real Source	Books	60
	37	Between Up	Toys	58
	17	Everything Plant	Books	58
	48	Group But	Clothing	58
	35	Fire Often	Electronics	58
	9	Common Study	Toys	57
	31	Series Page	Electronics	54

Q6. List all payments including those with no matching orders (RIGHT JOIN)**Columns Used:** *payments: all columns | orders: status*

```

SELECT pay.payment_id, pay.order_id, pay.payment_date,
       pay.amount_paid, pay.method, o.status AS order_status
FROM orders o
RIGHT JOIN payments pay ON o.order_id = pay.order_id;

```

Output:

	payment_id	order_id	payment_date	amount_paid	method	order_status
▶	1	1	2025-03-02 08:16:11	9414.28	Credit Card	Delivered
	2	2	2024-10-09 19:00:21	532.20	Net Banking	Shipped
	3	3	2025-05-08 00:55:27	5164.56	Credit Card	Cancelled
	4	4	2024-09-19 22:28:13	9469.78	UPI	Delivered
	5	5	2025-04-08 18:26:06	14501.86	UPI	Pending
	6	6	2024-10-25 08:16:59	31050.17	UPI	Cancelled
	7	7	2024-07-29 12:45:47	3043.67	UPI	Shipped
	8	8	2024-07-30 23:42:49	32714.06	Net Banking	Cancelled
	9	9	2025-06-10 17:36:25	24219.20	Net Banking	Pending
	10	10	2025-02-16 13:43:59	24342.52	Debit Card	Delivered
	11	11	2025-03-11 15:05:46	16196.13	Credit Card	Cancelled
	12	12	2025-01-15 14:00:53	9890.72	Credit Card	Shipped
	13	13	2025-05-12 14:01:29	9627.36	UPI	Cancelled

Q7. Combine data from three tables: customer, order, and payment (3-table JOIN)

Columns Used: *customers: name, email | orders: order_id, order_date, status, total_amount | payments: method, amount_paid, payment_date*

```
SELECT c.name AS customer_name, c.email, o.order_id, o.order_date, o.status,
       o.total_amount, pay.method AS payment_method, pay.amount_paid,
       pay.payment_date
FROM customers c
INNER JOIN orders o   ON c.customer_id = o.customer_id
INNER JOIN payments pay ON o.order_id   = pay.order_id
ORDER BY o.order_date DESC;
```

Output:

	customer_name	email	order_id	order_date	status	total_amount	payment_method	amount_paid	payment_date
▶	Shawn Hill	user10@example.com	62	2025-06-13 23:21:01	Cancelled	35723.17	Credit Card	35723.17	2025-06-13 23:45:01
	Amanda Bright	user19@example.com	173	2025-06-13 10:58:21	Shipped	6875.62	UPI	6875.62	2025-06-13 11:43:21
	Jason Thompson	user9@example.com	356	2025-06-11 13:30:19	Pending	11584.94	Net Banking	11584.94	2025-06-11 14:21:19
	Austin Flores	user15@example.com	286	2025-06-11 08:36:00	Pending	26952.09	UPI	26952.09	2025-06-11 08:47:00
	Deborah Arias	user14@example.com	308	2025-06-11 03:03:58	Cancelled	36411.35	Debit Card	36411.35	2025-06-11 03:45:58
	Thomas Owens	user1@example.com	127	2025-06-10 18:01:32	Pending	20160.64	Debit Card	160.64	2025-06-10 18:57:32
	Mark Duke	user6@example.com	9	2025-06-10 17:00:25	Pending	24219.20	Net Banking	24219.20	2025-06-10 17:36:25
	Cindy Hart	user25@example.com	313	2025-06-09 19:00:34	Cancelled	24245.67	Credit Card	24245.67	2025-06-09 19:41:34
	Kaitlin Richards	user3@example.com	134	2025-06-09 13:59:27	Pending	15483.53	Net Banking	15483.53	2025-06-09 14:20:27
	Deborah Arias	user14@example.com	166	2025-06-09 06:26:55	Pending	14925.84	Credit Card	14925.84	2025-06-09 06:59:55
	Randy Mooney	user17@example.com	256	2025-06-08 16:15:49	Delivered	40944.10	Debit Card	40944.10	2025-06-08 16:31:49
	Kara Zavala	user29@example.com	129	2025-06-06 21:11:17	Delivered	30752.72	Net Banking	30752.72	2025-06-06 22:10:17
	Deborah Arias	user14@example.com	231	2025-06-06 18:09:21	Cancelled	27402.85	Debit Card	27402.85	2025-06-06 18:57:21

Level 5: Subqueries (Inner Queries)

Q1. List all products priced above the average product price

Columns Used: *products: all columns, price*

```
SELECT * FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```

Output:

	product_id	name	category	price	stock_quantity	added_on
▶	7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
	2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
	10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
	4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
	35	Fire Often	Electronics	4734.89	198	2023-10-22 04:15:51
	5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
	19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
	41	Serious Recognize	Electronics	4523.10	112	2023-08-20 06:16:55
	38	Study Total	Toys	4413.68	31	2023-11-08 08:25:07
	20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
	33	Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17
	48	Group But	Clothing	4180.23	141	2023-11-29 01:38:12
	8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10

Q2. Find customers who have placed at least one order

Columns Used: *customers: all columns | orders: customer_id*

```
SELECT * FROM customers
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = customers.customer_id
);
```

Output:

	customer_id	name	email	phone	created_at
▶	1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
	2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
	3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
	4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
	5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
	6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
	7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
	8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
	9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
	10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
	11	Walter Jenkins	user11@example.com	536-329-0817x71	2023-10-26 03:12:30
	12	Mary Knight	user12@example.com	361-636-3802	2023-08-16 20:05:50
	13	Leslie Wilson	user13@example.com	+1-256-261-1984	2024-06-06 20:12:35

Q3. Show orders whose total amount is above the average for that customer**Columns Used:** *orders: all columns, customer_id, total_amount*

```
SELECT o.* FROM orders o
WHERE o.total_amount > (
    SELECT AVG(o2.total_amount) FROM orders o2
    WHERE o2.customer_id = o.customer_id
);
```

Output:

	order_id	customer_id	order_date	status	total_amount
▶	6	29	2024-10-25 07:33:59	Cancelled	31050.17
	8	19	2024-07-30 22:49:49	Cancelled	32714.06
	9	6	2025-06-10 17:00:25	Pending	24219.20
	10	28	2025-02-16 12:45:59	Delivered	24342.52
	14	1	2024-09-24 21:21:38	Shipped	15803.34
	16	5	2024-09-09 03:00:40	Shipped	22856.73
	18	13	2024-06-15 12:57:04	Cancelled	32001.24
	21	22	2025-03-12 18:53:46	Pending	25364.11
	22	20	2025-02-21 11:21:06	Shipped	21281.44
	24	6	2024-06-19 07:09:04	Pending	22882.19
	25	17	2024-07-24 12:56:24	Pending	23749.66
	26	24	2025-02-16 23:16:49	Delivered	35167.92
	29	7	2025-02-17 18:03:18	Delivered	26308.25

Q4. Display customers who haven't placed any orders**Columns Used:** *customers: all columns | orders: customer_id*

```
SELECT * FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

Output:

	customer_id	name	email	phone	created_at
*	NULL	NULL	NULL	NULL	NULL

Q5. Show products that were never ordered**Columns Used:** *products: all columns | order_items: product_id*

```
SELECT * FROM products p
WHERE NOT EXISTS (
    SELECT 1 FROM order_items oi
    WHERE oi.product_id = p.product_id
);
```

Output:

	product_id	name	category	price	stock_quantity	added_on
*	NULL	NULL	NULL	NULL	NULL	NULL

Q6. Show highest value order per customer**Columns Used:** *orders: customer_id, order_id, total_amount*

```
SELECT customer_id,  
       order_id,  
       total_amount  
FROM orders o  
WHERE total_amount = (  
    SELECT MAX(o2.total_amount)  
    FROM orders o2  
    WHERE o2.customer_id = o.customer_id  
)  
ORDER BY customer_id;
```

Output:

	customer_id	order_id	total_amount
▶	1	278	32015.16
	2	315	42056.04
	3	281	41679.11
	4	386	25747.34
	5	266	39921.78
	6	330	39003.19
	7	128	28589.04
	8	33	36147.09
	9	348	21414.44
	10	62	35723.17
	11	133	37129.82
	12	80	38656.26
	13	18	32001.24

Q7. Highest order per customer (including customer names)**Columns Used:** *orders: customer_id, order_id, total_amount* / *customers: name*

```

SELECT c.name AS customer_name,
       o.order_id,
       o.total_amount AS highest_order_amount
FROM orders o
JOIN customers c
  ON o.customer_id = c.customer_id
WHERE o.total_amount = (
  SELECT MAX(o2.total_amount)
  FROM orders o2
  WHERE o2.customer_id = o.customer_id
)
ORDER BY o.total_amount DESC;

```

Output:

	customer_name	order_id	highest_order_amount
▶	Deborah Arias	107	48042.06
	Joseph Stuart	393	44504.56
	Kara Zavala	309	43867.08
	Charles Grant	315	42056.04
	Kaitlin Richards	281	41679.11
	Randy Mooney	256	40944.10
	Cindy Hart	189	40872.29
	Austin Flores	243	40510.54
	David Allen	266	39921.78
	Jeffrey Bray	139	39857.19
	Megan Lee	327	39563.51
	Mark Duke	330	39003.19
	Victoria Hall	258	38836.44

Level 6: Set Operations

Q1. List all customers who have either placed an order or written a product review (UNION)

Columns Used: *customers: customer_id, name, email | orders: customer_id | product_reviews: customer_id*

```
SELECT customer_id, name, email FROM customers
WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders
    WHERE customer_id IS NOT NULL)
UNION
SELECT customer_id, name, email FROM customers
WHERE customer_id IN (SELECT DISTINCT customer_id FROM product_reviews);
```

Output:

	customer_id	name	email
▶	1	Thomas Owens	user1@example.com
	2	Charles Grant	user2@example.com
	3	Kaitlin Richards	user3@example.com
	4	Christina Williams	user4@example.com
	5	David Allen	user5@example.com
	6	Mark Duke	user6@example.com
	7	Briana Wright	user7@example.com
	8	John Bryan	user8@example.com
	9	Jason Thompson	user9@example.com
	10	Shawn Hill	user10@example.com
	11	Walter Jenkins	user11@example.com
	12	Mary Knight	user12@example.com
	13	Leslie Wilson	user13@example.com
	14	Deborah Arias	user14@example.com

Q2. List customers who have placed an order AND reviewed a product (INTERSECT equivalent)

Columns Used: *customers: customer_id, name, email | orders: customer_id | product_reviews: customer_id*

```
SELECT DISTINCT c.customer_id, c.name, c.email
FROM customers c
WHERE c.customer_id IN (
    SELECT DISTINCT customer_id FROM orders WHERE customer_id IS NOT NULL)
AND c.customer_id IN (
    SELECT DISTINCT customer_id FROM product_reviews);
```

Output:

	customer_id	name	email
▶	1	Thomas Owens	user1@example.com
	2	Charles Grant	user2@example.com
	4	Christina Williams	user4@example.com
	6	Mark Duke	user6@example.com
	7	Briana Wright	user7@example.com
	9	Jason Thompson	user9@example.com
	10	Shawn Hill	user10@example.com
	11	Walter Jenkins	user11@example.com
	13	Leslie Wilson	user13@example.com
	14	Deborah Arias	user14@example.com
	15	Austin Flores	user15@example.com
	17	Randy Mooney	user17@example.com
	19	Amanda Bright	user19@example.com

7. Project Analysis

The following observations and insights were derived from the SQL queries executed on the Retail database:

7.1 Data Volume

Column	Data Type	Description
Customers	30 records	30 unique customers registered
Products	50 records	50 products across 5 categories
Orders	400 records	400 orders placed over the analysis period
Order Items	1,201 records	Line items linking orders to products
Payments	400 records	One payment record per order
Product Reviews	50 records	Customer ratings for various products

7.2 Key Findings

Customer Behaviour

All 30 registered customers have at least one order, indicating a fully active customer base in this dataset. No orphaned customer records were found.

Product Categories

Products span five categories: Electronics, Clothing, Home, Books, and Toys. Electronics and Clothing feature the highest price-point items (above ₹4,000), while Home and Toys cover a wider price range.

Order Status Distribution

Orders are distributed across four statuses — Delivered, Shipped, Pending, and Cancelled — providing a realistic simulation of an active fulfilment pipeline. Cancelled orders still carry payment records, which is relevant for refund analysis.

Payment Methods

Four payment modes are represented: UPI, Credit Card, Debit Card, and Net Banking. The Level 3 aggregation query reveals the total revenue collected per method, useful for finance reporting.

Query Complexity Coverage

The 36 queries across 6 levels provide comprehensive coverage: basic SELECT/WHERE/ORDER, formatting with CONCAT and DATE(), GROUP BY aggregations, all major JOIN types (INNER, LEFT, RIGHT, 3-table), correlated subqueries with EXISTS/NOT EXISTS, window functions (ROW_NUMBER), and UNION set operations.

8. Conclusion

This SQL Mini Project successfully demonstrates the application of Structured Query Language across a realistic retail business scenario. The six-level query structure progressively builds from foundational SELECT statements to advanced analytical queries involving window functions and correlated subqueries.

The Retail database — comprising 6 interrelated tables and realistic sample data — serves as an effective testbed for a broad range of SQL concepts. The queries cover the core competencies expected in both academic and professional data analysis contexts.

All queries are error-free, logically structured, and annotated with their purpose and the specific columns they operate on. The project can be extended further with views, stored procedures, indexes, and trigger-based automation as next steps.